ЛАБОРАТОРНА РОБОТА № 5
Арифметичні та логічні функції в мові Verilog

Мета роботи: освоїти лексику мови Verilog для виконання логічних та арифметичних операцій. Закріпити практичні навички зі створення складних проектів. Створити проект простого калькулятора.

Теоретичні відомості

Ознайомимося з основними арифметичними і логічними операторами мови Verilog.

Додавання і віднімання.

Ось приклад модуля, який одночасно і складає і віднімає два числа. Тут вхідні операнди у нас 8-бітові, а результат 9-бітовий. Verilog коректно згенерує біт переносу (carry bit) і помістить його в дев'ятий біт вихідного результату. З погляду Verilog, вхідні операнди беззнакові. Якщо потрібна знакова арифметика, то про це потрібно окремо подбати.

```
module simple_add_sub ( operandA, operandB,
out_sum, out_dif);
// два вхідних 8-бітових операнди
    input [7:0] operandA, operandB;
 /* виходи для арифметичних операцій мають
додатковий  9-й біт переповнення */
   output [8:0] out_sum, out_dif;
   assign out_sum = operandA + operandB; // додавання
   assign out_dif = operandA - operandB; // віднімання
endmodule
```

Бітові логічні операції.

Бітові операції в Verilog виглядають так само, як і в мові С. Кожен біт результату обчислюється окремо відповідно бітам операндів.

Нижче наведено приклад бітових логічних операцій мовою Verilog.

```
module simple_bit_logic ( operandA, operandB, out_bit_and,
out_bit_or, out_bit_xor, out_bit_not);
   input [7:0] operandA, operandB;
   output [7:0] out_bit_and, out_bit_or, out_bit_xor,
out_bit_not;
   assign out_bit_and = operandA & operandB; // І
   assign out_bit_or = operandA | operandB; // АБО
   assign out_bit_xor = operandA ^ operandB; // виключне
АБО
```

```
    assign out_bit_not = ~operandA; // НЕ
endmodule
```

Булеві логічні операції.

Булеві логічні оператори відрізняються від бітових операцій. Так само, як і в мові С, тут значення всієї шини розглядається як TRUE («1») якщо хоча б один біт в шині одиниця або FALSE («0»), якщо всі біти шини – нуль. Результат виходить завжди однобітовий незалежно від розрядності операндів.

```
module simple_bit_logic ( operandA, operandB,
out_bool_and, out_bool_or, out_bool_not);
    input [7:0] operandA, operandB;
    output out_bool_and, out_bool_or, out_bool_not;
    assign out_bool_and = operandA && operandB; // І
    assign out_bit_or = operandA || operandB; // АБО
    assign out_bit_not = !operandA; // НЕ
endmodule
```

Оператори редукції.

Verilog має оператори редукції. Ці оператори дозволяють виконувати операції між бітами всередині однієї шини. Так, можна визначити, чи всі біти в шині дорівнюють одиниці (& bus), або чи є в шині хоча б одна одиниця (| bus). Наприклад:

```
module simple_reduction_logic ( operandA,
out_reduction_and, out_reduction_or, out_reduction_xor);
    input [7:0] operandA;
    output out_reduction_and, out_reduction_or,
out_reduction_xor;
    assign out_reduction_and = &operandA;
    assign out_reduction_or = |operandA;
    assign out_reduction_xor = ^operandA;
endmodule
```

Також корисні оператори редукції: ~ | operandA означає, що в шині немає одиниць, ~ & operandA означає, що деякі біти в шині дорівнюють нулю.

Оператор умовного вибору.

Мова С має оператор «? : ». З його допомогою можна вибрати одне значення з двох за результатом логічного виразу. У Verilog теж є подібний оператор, він фактично реалізує мультиплексор. У цьому прикладі на

виході мультиплексора виявиться значення operandA, якщо сигнал sel_in одиниця. І навпаки. Якщо вхідний сигнал sel_in дорівнює нулю, то на виході мультиплексора буде значення operandB.

```
module simple_mux ( operandA, operandB, sel_in, out_mux );

    input [7:0] operandA, operandB;
    input sel_in;  //вхідний сигнал селектора
    output [7:0] out_mux;
    assign out_mux = sel_in ? operandA : operandB;

endmodule
```

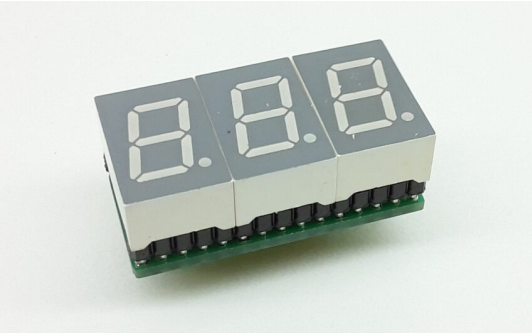
Оператори порівняння.

Змінні отримають значення логічної одиниці, якщо порівняння підтвердилось, і – логічного нуля, якщо ні.

```
module simple_compare ( operandA, operandB, out_eq,
out_ne, out_gt, out_lt, out_ge, out_le);

    input [7:0] operandA, operandB;
    //виходи операцій порівняння
    output out_eq, out_ne, out_gt, out_lt, out_ge, out_le;
    assign out_eq = operandA == operandB; //рівно
    assign out_ne = operandA != operandB; //нерівно
    assign out_ge = operandA >= operandB; //більше або
рівно
    assign out_le = operandA <= operandB; //менше або
рівно
    assign out_gt = operandA > operandB; //більше
    assign out_lt = operandA < operandB; //менше

endmodule
```

Завдання до роботи

1. Створіть проект простого мультиплексора. Запрограмуйте та перевірте роботу.
2. Створіть проект простого калькулятора з виводом результату в десятковій формі на уявні 7-сегментні індикатори (див. рис.). Запрограмуйте та перевірте роботу.